

AI Dark Factory — Architecture Whitepaper

An autonomous L4 code generation platform. Requirements enter, production-quality code comes out. This document explains how.

Author	Kevin Quon — linkedin.com/in/kwkwan00
Backend	FastAPI + Uvicorn (Python 3.12+)
Frontend	React 18 + Vite + TypeScript
Agent runtime	LangGraph swarm (create_swarm) + Claude Agent SDK
Knowledge store	Neo4j (graph + procedural memory)
Vector store	Qdrant (text-embedding-3-large, 3072 dim)
Metrics	Prometheus + Grafana + optional Postgres forensic store
Protocol	AG-UI Server-Sent Events for real-time UI streaming
Storage	Pluggable local, S3, or replicated (local + S3) storage — run-ID-scoped layout with input/, requirements/, specs/, output/ subfolders. Auto-replicates to S3 when S3_BUCKET is configured alongside local storage.
Agentic level	L4 — Fully Autonomous / Explorer

Business Value

What this platform delivers — in plain terms

AI Dark Factory takes the gap between “*here is what we want*” and “*here is running, tested code*” and closes it autonomously. A human writes requirements; the system produces production-quality code, tests, evaluation reports, and a reconciled integration — without human intervention between phases. Every run teaches the platform something that makes the next run faster and better.

OUTCOMES

Faster time-to-code	Requirements become working, tested code in a single invocation. Engineers supervise outcomes instead of typing them.
Continuous improvement	Procedural memory persists across runs. The system remembers what worked, what broke, and how it was fixed — and applies those lessons automatically next time.
Predictable quality	Every spec and every artifact is evaluated by an independent LLM judge against an explicit rubric. No artifact ships without a score.
Full cost visibility	Every LLM call, token count, tool invocation, and eval result is recorded. Operators see exactly what a run cost, where the time went, and which features drove the spend.
Graceful degradation	A single failing feature does not take down a run. Reconciliation failures do not block delivery. Cancellation is clean and leaves no orphaned state.
Low operational overhead	Runtime settings are live-mutable from the UI — parallelism, handoff limits, model selection, eval thresholds — all without redeploying.
Auditable decisions	Every agent action, handoff, tool call, and evaluation is logged, timestamped, and accessible in the Run Detail popup. Every run leaves a complete forensic trail.

CORE FEATURES FOR BUSINESS

Requirements ingestion	Drop any mix of Markdown, JSON, YAML, OpenSpec, Word, Excel, PDF, HTML, XML, RTF, CSV, or transcript files. Native formats parse directly; rich business documents are routed through a clean-context Claude Agent SDK invocation that extracts discrete requirements per document.
Automatic specification	Requirements become formal specs with acceptance criteria, dependencies, and capability tags — scored for correctness, coherence, instruction-following, and safety.
Per-feature codegen swarms	Four specialised agents (Planner, Coder, Reviewer, Tester) collaborate per feature. Features run in parallel within dependency layers.
Cross-feature reconciliation	After every run, a final polishing pass reviews the entire output, fixes broken integrations, runs the test suite, and produces a reconciliation report.
Cross-browser E2E validation	After reconciliation, a Playwright smoke-test pass exercises the generated application in chromium, firefox, and webkit. Test results, failure screenshots, and a browsable HTML report are surfaced in the Run Detail popup so the operator can verify the app actually runs in a real browser — not just that it compiled.
Run history	Every run is persisted with spec counts, pass rate, mean eval scores, worst features, and full event timeline. The Manufacture tab shows recent runs with live polling, status badges (success/partial/cancelled/error), and an actions menu for deletion. Click any run to open a detail popup with tabs for: Agent Log, Metrics, Evaluations, Episodes, Output (with Download ZIP), Compare (side-by-side against another run with file diffs), and Traceability (requirements → specs → files → tests → evals matrix with interactive dependency graph).
Requirements Refinery	A dedicated Refine tab lets operators transform raw requirements into a more detailed, structured, and actionable set. Powered by concurrent deep agents (OpenAI Responses API), it analyzes each requirement for clarity, completeness, and testability, discovers inter-requirement relationships, suggests spec decompositions, and proposes reusable strategy/pattern memories. A reconciliation phase detects duplicates, coherence issues, and priority inversions across the full set. Results are persisted to S3/local storage with full history and export to ZIP.
Live observability	Real-time agent logs with color-coded event badges, comprehensive metrics dashboards, Prometheus + Grafana integration, per-run cost rollups, and per-run agent logs with expandable JSON payloads. Know what the system is doing and what it has done.
Safe cancellation	A single Cancel button cleanly stops any running pipeline at the next checkpoint, with no orphaned state or half-written records.
Deep agent resilience	Claude Agent SDK subprocess crashes are caught and converted to soft errors via <code>_safe_tool_deep_agent</code> . Stderr is buffered for diagnostics, and <code>--debug-to-stderr</code> can be enabled via <code>DEEP_AGENT_DEBUG_STDERR</code> for investigating silent exits. A regression test verifies all tool-decorated deep agents use the safe wrapper.

WHO THIS IS FOR

Engineering teams that want to let a coordinated agent swarm handle routine feature delivery while humans focus on architectural direction, requirements quality, and review of the final output. Not a replacement for engineers — a force multiplier for them.

1. Design Philosophy

AI Dark Factory is built on a single premise: **a coordinated swarm of specialised agents with persistent memory produces higher-quality code than a single monolithic agent with a bigger context window**. Every architectural choice in this system flows from that premise.

The pipeline is decomposed into seven well-defined phases. Each phase has a narrow contract, its own evaluation criteria, and a clean failure boundary. Agents operate in tight feedback loops with external judges (DeepEval + GPT), write what they learn into a shared memory graph, and benefit from every prior run — even partial ones.

WHY SPECIALISATION BEATS SCALE

The natural instinct when a model underperforms is to reach for a larger model with a bigger context window. Dark Factory takes the opposite bet: keep each agent's context narrow and its responsibility singular, then coordinate. A Planner that only ever reads specs and eval history stays focused. A Coder that only writes code — informed by RAG-retrieved patterns and explicit mistakes from the Reviewer — accumulates a tighter internal model of what good code looks like for this codebase. A Reviewer that only evaluates and records catches problems the Coder cannot self-diagnose. No single agent needs to juggle the full problem space; the swarm does.

This mirrors how high-performing engineering teams actually work. A senior engineer does not also run quality assurance, write the design doc, and manage the deployment — those roles exist because division of labour produces better outcomes than a single generalist doing everything. The swarm encodes that intuition in software.

MEMORY AS INFRASTRUCTURE, NOT AN AFTERTHOUGHT

Most agent systems treat memory as a convenience feature bolted on after the core loop is working. Dark Factory treats it as load-bearing infrastructure. Every agent reads from the shared Neo4j + Qdrant memory graph before its first tool call, and every agent writes back after its last. The memory graph is not a log — it is a living, scored, decaying knowledge base. Lessons that prove useful accumulate relevance and surface first. Lessons that fail in practice decay and eventually prune themselves. The system does not just remember; it forgets intelligently.

The episodic tier adds temporal reasoning on top of semantic generalisation. Semantic memory answers "what should I do?" Episodic memory answers "what actually happened the last three times I was in this exact situation?" For a Planner choosing between strategies, the episodic record is often more actionable than a generalised pattern hit — it carries the full trajectory, including the wrong turns and the recovery.

EVALUATION AS A FIRST-CLASS DEPENDENCY

Evaluation is not a reporting step that runs after the pipeline finishes. It is wired into the pipeline as a dependency that gates phase transitions, drives memory feedback, and triggers adaptive strategy switches mid-run. A spec that scores below threshold does not advance to the graph write. A feature swarm whose pass rate drops below the layer threshold causes the orchestrator to tighten handoff budgets and force the SDK path for remaining features. Evaluation results do not sit in a dashboard waiting to be read — they change what happens next.

EXTERNAL JUDGING

Evaluation uses DeepEval with GPT as the judge, intentionally separate from the Anthropic models used for codegen. A model cannot reliably judge its own output. Using a different model family as judge introduces the adversarial distance needed for scores to be meaningful.

FAILURE ISOLATION AND GRACEFUL DEGRADATION

Every failure boundary in the system is explicit and narrow. A single feature swarm failing does not abort the run — the orchestrator records the failure, emits an incident, and continues to the next layer. Phase 5 reconciliation failing does not block delivery — the pipeline ships the unpolished feature output and logs the reconciliation error as an incident. Phase 6 E2E validation failing does not remove the output from the operator's hands — they see the E2E report and decide whether the delivery is ship-ready themselves.

This is a deliberate design choice, not a compromise. Automated systems that hard-fail on any error are fragile in production. A system that delivers partial results with full transparency about what failed — and why — is more useful than one that delivers nothing and requires a human to restart from scratch.

OBSERVABILITY BY CONSTRUCTION

Observability is not instrumented as a separate concern — it is woven into the execution path. Every phase, tool call, LLM invocation, handoff, eval result, and incident flows through the same instrumentation layer that emits to Prometheus, writes to Postgres, and publishes to the ProgressBroker simultaneously. The three sinks are independent: a Postgres outage does not silence the Prometheus counters; a ProgressBroker subscriber crash does not interrupt the forensic row store. Operators can watch a run live in the Agent Logs tab, query it historically in Run Detail, and alert on it via Grafana — all from the same underlying event stream.

CORE TENETS

Specialisation over scale	Four small agents with distinct roles beat one big agent told to do everything. Narrow context produces tighter reasoning.
Memory is first-class	Procedural memory is a scored, decaying graph — not a cache. Every agent reads before acting, writes after. Lessons that work accumulate; lessons that fail decay.
Evaluation is a dependency	LLM judges (DeepEval + GPT) run on every spec and artifact. Scores gate phase transitions, drive memory feedback, and trigger mid-run strategy switches.
Failure isolation	Every failure boundary is explicit and narrow. A failed feature, a crashed reconciliation, or a broken E2E pass never aborts the run — partial delivery with full transparency beats no delivery.
Best-effort polishing	Phase 5 reconciliation and Phase 6 E2E are intentionally wrapped in broad try/except. They polish and validate; they never gatekeep.
Observability by construction	Every tool call, phase, handoff, and incident flows through three independent sinks (Prometheus, Postgres, ProgressBroker) simultaneously. No single telemetry failure silences the others.

2. System Overview

How the pieces fit together

The runtime is a single-process FastAPI application that hosts both the REST API and the React single-page application (served from `src/dark_factory/api/static/`). A background daemon asyncio loop (the `BackgroundLoop` singleton) runs every Claude Agent SDK invocation on a dedicated thread so that subprocess cleanup callbacks always have a valid loop to land on — this eliminates a whole class of “Event loop is closed” errors that plagued earlier iterations.

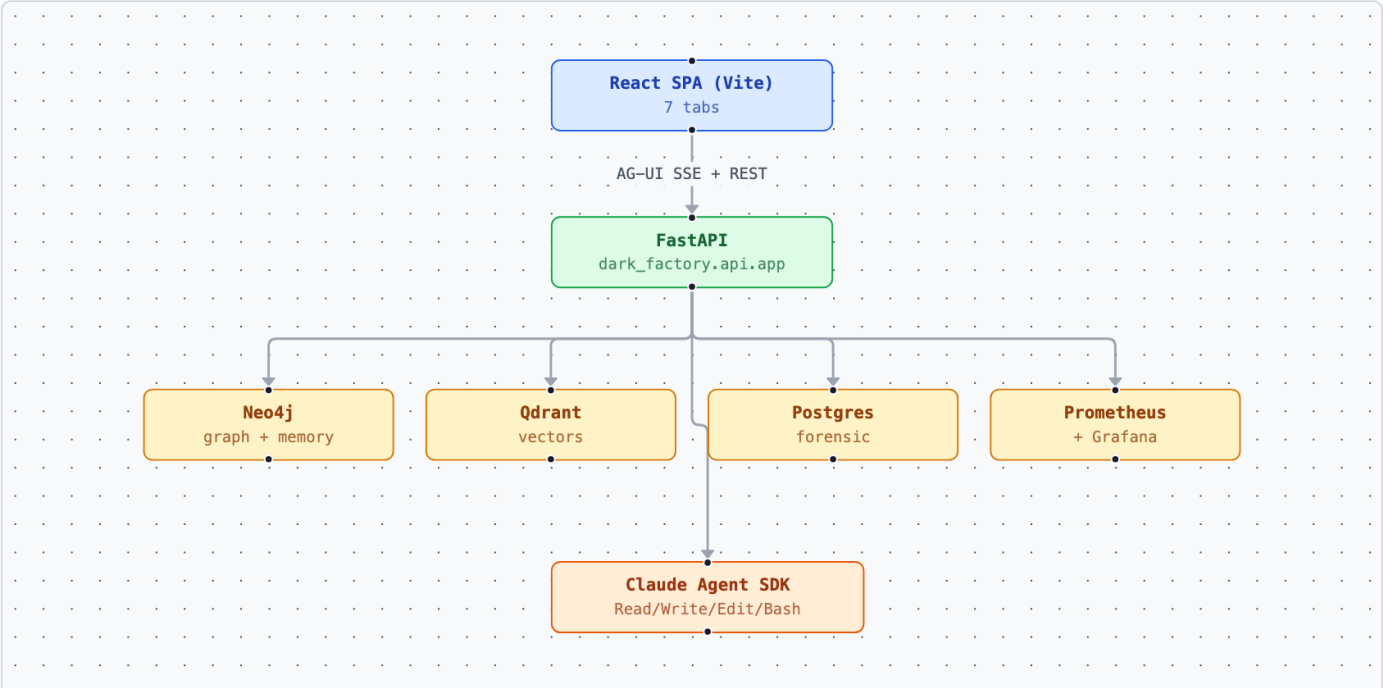


Figure 1 — High-level system topology

Agents communicate with the outside world through a small set of services that are each replaceable and individually health-checked:

Neo4j	Primary store for specs, requirements, dependencies, runs, and procedural memory nodes.
-------	---

Qdrant	Hybrid RAG: similar-spec / similar-code / memory semantic search via text-embedding-3-large.
Postgres	Optional forensic row store for LLM calls, tool calls, eval results, incidents, progress events.
Prometheus	Always-on in-process counters and histograms — zero external dependency.
Grafana	Pre-provisioned dashboards over the Prometheus scrape target.
Claude Agent SDK	File I/O + Bash tools for Phase 5 reconciliation and Phase 6 E2E validation. Deep-agent tools (codegen, analysis, test gen) use direct Anthropic API calls.
Storage backend	Pluggable local filesystem, S3, or replicated (local + S3) storage. When STORAGE_BACKEND=local and S3_BUCKET is set, a ReplicatedStorage backend auto-writes to both local disk (fast pipeline I/O) and S3 (durable off-host persistence). S3 failures are best-effort and never block the pipeline. A catch-all sync in the pipeline's finally block ensures the full run directory reaches S3 even on mid-pipeline crashes.

3. The Seven-Phase Pipeline

From requirements to production-quality code

A single `POST /api/agent/run` call drives the entire pipeline. The request returns immediately; subsequent events stream to the browser over `GET /api/agent/events` using the AG-UI Server-Sent Events format.

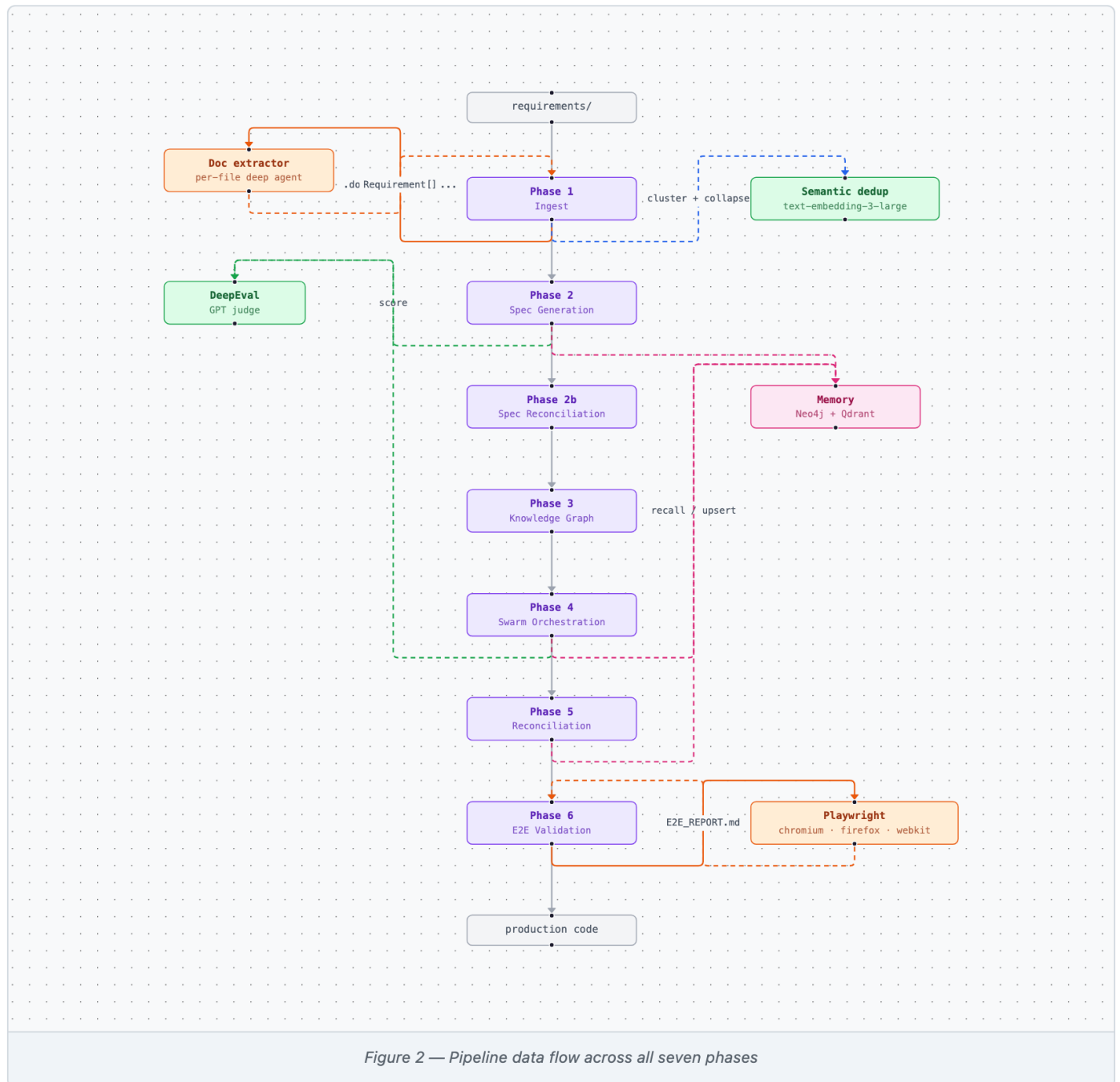


Figure 2 — Pipeline data flow across all seven phases

PHASE 1 — INGEST

`IngestStage` parses a requirements directory into `Requirement` domain models. Two tiers of input are supported. **Native formats** (`.md`, `.txt`, `.json`, `.yaml`, OpenSpec `specs/` trees) are parsed directly; OpenSpec inputs extract WHEN/THEN scenarios into structured acceptance criteria and large text documents go through an LLM splitter that breaks each discrete testable requirement into its own entry.

Rich business documents (Word, Excel, PowerPoint, PDF, HTML, XML, RTF, CSV, transcripts) are routed through a **clean-context Claude Agent SDK invocation** per file. Each invocation runs in a fresh subprocess with its cwd set to the upload directory and has access to `Read / Write / Edit / Glob / Grep / Bash`. The agent reads the document with the appropriate Python library (`python-docx`, `openpyxl`, `pypdf`, `beautifulsoup4`, `striptrtf`, `lxml`), extracts discrete testable requirements, and writes a staging JSON file that the ingest stage loads back into `Requirement` models. Because each document gets its own isolated agent context, raw meeting-transcript noise never pollutes the main pipeline's LLM context.

After all files are parsed, the stage runs a **semantic deduplication pass**. Every requirement is embedded with `text-embedding-3-large`; near-duplicates (cosine similarity $\geq$ `requirement_dedup_threshold`, default 0.90) are clustered and collapsed into a single canonical entry — preferring the highest-priority + most detailed member, breaking ties by original position so re-runs are deterministic. Tags from every merged requirement are unioned onto the canonical so source-document attribution is never lost. This is a *correctness guarantee*: a corpus assembled from a meeting transcript, a Word brief, and a spreadsheet routinely describes the same requirement multiple ways, and without dedup the Spec stage would burn LLM budget generating duplicate specs for each. Embedding outages fall back to the un-deduped list rather than blocking the pipeline.

OBSERVABILITY

When dedup merges anything, the Agent Logs tab shows a `requirements_deduped` event with the input/ output counts and a preview of each merge cluster ("*kept X, merged Y, Z*"). The first 5 clusters are listed inline; larger dedups link to the structured log.

PHASE 2 — SPEC GENERATION

Each requirement optionally runs through a **planner decomposition** step that splits it into multiple granular sub-specs. Every sub-spec then enters a dedicated **architect → critic → refine** loop that iterates until either the DeepEval score crosses `spec_eval_threshold` or the handoff budget is exhausted.

An **early-exit preflight** queries Neo4j via `IMPLEMENTS` edges before the planner even runs. If every requirement already has specs, the entire decomposition + refinement loop is skipped — re-runs on unchanged inputs complete in a single Neo4j query with zero LLM spend. Partial coverage falls through to the normal path, reusing existing specs and only generating missing ones.

DEEPEVAL RUBRIC

Every spec is scored on four GEval metrics: Correctness, Coherence, Instruction Following (does it match the OpenSpec shape?), and Safety & Ethics. GPT acts as judge. Scores feed both the early-exit threshold and the adaptive memory decay in Phase 4.

PHASE 2B — SPEC RECONCILIATION

After spec generation and before the graph write, a reconciliation stage validates and repairs the spec dependency graph. A **deterministic pass** strips phantom references, detects and breaks circular dependencies, flags uncovered requirements, and identifies disconnected capability islands. An **LLM-assisted pass** then reviews spec descriptions, acceptance criteria, and WHEN/THEN scenarios to surface implicit dependencies the planner missed, fix requirement coverage gaps, and correct capability groupings. The LLM pass is sandboxed — cycles and phantoms are re-checked after patches are applied so the LLM cannot corrupt the graph.

CAPABILITY ISLANDS

Specs that share a `capability` but have no dependency path between them are flagged as "islands." This usually means a missing dependency edge (the data model spec should be a dependency of the API handler spec) or a wrong capability assignment. The reconciliation surfaces these for both the LLM and the operator.

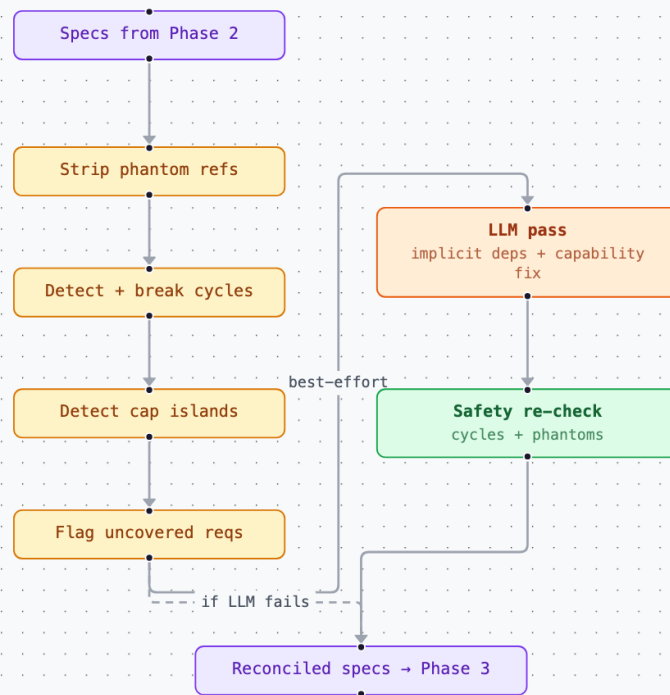


Figure — Spec reconciliation flow. Deterministic checks (left) always run; LLM pass (right) is best-effort with safety re-check.

PHASE 3 — KNOWLEDGE GRAPH

Specs and requirements are persisted to Neo4j with `IMPLEMENTS` and `DEPENDS_ON` relationships. Specs are simultaneously auto-indexed into the Qdrant `dark_factory_specs` collection with enriched metadata (eval scores, attempts, scenarios, dependencies) so that downstream Coder agents can pull in semantically-similar work from other features as RAG context.

PHASE 4 — SWARM ORCHESTRATION

The orchestrator groups specs by `capability` and uses Tarjan's Strongly Connected Components algorithm to compute a **cycle-tolerant topological order**. Features within the same dependency layer run in parallel, bounded by `max_parallel_features`. Each feature spawns an isolated LangGraph swarm with four specialised agents (see Section 4).

After each layer, the orchestrator reviews the layer's pass rate. If it drops below threshold, the Coder is forced onto the SDK path for all remaining features and the handoff budget is tightened. When performance recovers on a later layer, the overrides relax. This is the **adaptive strategy override** — a cheap but powerful lever that prevents a degrading run from eating its whole budget.

PHASE 5 — RECONCILIATION

The youngest and arguably most important phase. A **single extended Claude Agent SDK invocation** runs over the full run output directory with the complete `Read / Write / Edit / Glob / Grep / Bash` tool set. It follows a rigid six-step checklist: inventory → review → fix → validate → iterate → report. Because it can see every feature at once, it catches the class of bugs that per-feature swarms structurally cannot — broken cross-feature imports, inconsistent API shapes between frontend and backend, missing `main.py` / `package.json` / `requirements.txt` glue, and runtime-only failures that manifest when the pieces are assembled.

BEST-EFFORT CONTRACT

Phase 5 is intentionally wrapped in a broad try/except. Timeouts, crashes, and SDK failures are logged, recorded as incidents, surfaced in the Run Detail popup — and then *swallowed*. The pipeline always delivers feature output, even if reconciliation couldn't run. Polishing, not gatekeeping.

PHASE 6 — END-TO-END VALIDATION

The final phase. After reconciliation completes with a `clean` status (not just “not error” — `partial`, `skipped`, and `error` all gate this phase off), a second **clean-context Claude Agent SDK invocation** runs a **Playwright cross-browser smoke test suite** against the generated application. The agent follows a rigid six-step checklist — detect → install → plan → write → run → report — and has access to `Read / Write / Edit / Glob / Grep / Bash`. Browser binaries for `chromium`, `firefox`, and `webkit` are pre-installed in the Docker image at `/ms-playwright`, so the agent does not have to pay the download cost on every run.

In Step 1 the agent checks whether the output is actually a web application (package manifests, `Dockerfile` ports, HTML entry points, framework markers); if not, it writes a `skipped` report and stops. Step 3 picks 3–8 user-facing acceptance criteria from the specs and generates a minimal `e2e/smoke.spec.ts` plus a `playwright.config.ts` that enables every browser in the matrix. Step 5 starts the server in the background with a shell trap so it is always killed on exit, polls the health endpoint for readiness, and runs the suite with the `line` + `html` reporters. Step 6 writes `E2E_REPORT.md` at the run output root with a per-test / per-browser table, failure reasons, and an `Overall status` of `pass` / `partial` / `broken`.

The Playwright HTML report is captured to `e2e_artifacts/html-report/` and any failure screenshots end up under `e2e_artifacts/` — both are surfaced inline in the Run Detail popup so the operator can click through a failing test without leaving the UI. Per-browser test counts are fanned out to the `dark_factory_e2e_tests_total{browser, status}` Prometheus counter for dashboards that need to answer “is webkit the flaky one?”.

BEST-EFFORT CONTRACT

Phase 6 inherits the same policy as Phase 5: agent crashes, timeouts, server startup failures, and flaky tests are all logged, recorded as incidents, and then *swallowed*. A broken E2E pass never fails the run. The pipeline always delivers the reconciled feature output, and the operator reads the E2E report to decide whether the delivery is ship-ready. When E2E is skipped due to reconciliation status, a skip-reason text event is emitted to the Agent Log so operators see exactly why.

3.5 Deep Agents

How Claude Agent SDK subprocess agents are used throughout the pipeline

Dark Factory makes extensive use of the **Claude Agent SDK** to spawn isolated, clean-context subprocess agents at several points in the pipeline. Unlike the LangGraph swarm agents — which share a persistent graph state across handoffs — deep agents run in a completely fresh context, with their own working directory and a full file-system tool set: `Read / Write / Edit / Glob / Grep / Bash`. Each invocation starts with zero memory of previous runs and exits cleanly when its task is complete. This isolation is a feature, not a limitation: it means a deep agent's context is never polluted by earlier pipeline state, and it can operate directly on the filesystem without coordination overhead.

WHERE DEEP AGENTS ARE USED

Phase 1 — Document extraction

Rich business documents (Word, Excel, PDF, HTML, XML, RTF, CSV, transcripts) are routed to a per-file deep agent. Each invocation runs in a fresh subprocess with its cwd set to the upload directory, reads the document with the appropriate Python library, and writes a staging JSON file of discrete requirements. Because each document gets its own isolated context, raw meeting-transcript noise never pollutes the main pipeline's LLM context.

Phase 4 — Code generation

The Coder agent in the LangGraph swarm can delegate to a `claude_agent_codegen` deep agent for complex implementation tasks. The deep agent has full `Read / Write / Edit / Glob / Grep / Bash` access to the run output directory, so it can create files, run the linter, check imports, and iterate — all within a single SDK invocation. The swarm Coder receives the result as a structured tool response and continues.

Phase 4 — Code review and test generation

Specialised deep agents handle dependency analysis, risk review, security review, performance review, compliance review, and unit / integration / edge-case test generation. Each is a separate `@tool`-decorated function backed by its own SDK invocation, scoped to the files it needs to read and the output location it writes to.

Phase 5 — Reconciliation

A single extended deep agent runs over the full run output directory after all feature swarms complete. It follows a rigid six-step checklist — inventory → review → fix → validate → iterate → report — and is the only agent in the pipeline that can see every feature's output simultaneously. This gives it the authority to fix cross-feature import breakage, inconsistent API shapes, and missing glue files that per-feature swarms structurally cannot detect.

Phase 6 — E2E validation

A second extended deep agent runs after reconciliation to execute a Playwright cross-browser smoke test suite. It detects whether the output is a web application, writes a smoke.spec.ts against the acceptance criteria, starts the server, and runs the test suite across chromium, firefox, and webkit — all within a single subprocess invocation.

WHY SUBPROCESS ISOLATION MATTERS

Each deep agent runs as a Node.js subprocess managed by the `BackgroundLoop` singleton — a daemon asyncio event loop that ensures subprocess cleanup callbacks always have a valid loop to land on. Isolation delivers three concrete benefits: the agent's file-system tools operate on real disk paths with no translation layer; its LLM context contains exactly what its task needs and nothing else; and its exit is clean — the subprocess terminates, releasing all resources, before the calling swarm agent processes the result.

CLEAN-CONTEXT ADVANTAGE

The reconciliation and E2E agents deliberately start with no knowledge of how individual features were built. They evaluate the output as a whole — the same way an engineer picking up an unfamiliar codebase would. This cold-start perspective is what makes them effective at catching integration problems that the feature authors are too close to see.

4. Swarm Mechanics

Four agents, one feature, bounded handoffs

WHY A SWARM?

The canonical alternative to a swarm is a single large-context agent given the full spec and told to produce code, tests, and a review in one pass. This approach has a well-documented failure mode: as the context fills — with the spec, the code draft, the review notes, the revised draft, the test suite — the model's attention dilutes. Early instructions get overweighted relative to late corrections. The agent loses the thread of its own critique. Quality degrades as a function of context length, not task complexity.

The swarm sidesteps this entirely. Each agent receives only the context relevant to its role. The Coder's context is the spec, the RAG results, and the recalled patterns — not the Reviewer's notes from three rounds ago. The Reviewer's context is the generated code and the evaluation rubric — not the Coder's internal reasoning about why it made a particular choice. Each handoff resets the active context to exactly what the next agent needs. This means a 50-handoff swarm maintains sharp attention at every step, rather than a single agent running out of effective context after the first revision cycle.

There is a second benefit that compounds over time: **specialisation enables targeted memory**. Because the Reviewer always writes Mistake nodes and the Coder always reads Pattern nodes, the memory graph accumulates role-specific signal. A Reviewer mistake from run 3 surfaces in the Coder's recall on run 7 — not as noise in a general conversation history, but as a structured, scored, typed memory node that the retrieval system can rank against the current feature's capability tag. A monolithic agent has no clean boundary between "what I generated" and "what I evaluated" — the swarm makes that boundary explicit, and the memory system exploits it.

THE COMPOUNDING ARGUMENT

A single-agent system plateaus at whatever quality level its context window and base model allow. A swarm with persistent memory does not plateau — each run deposits lessons that make the next run start from a higher baseline. The quality ceiling of the system rises with usage.

Each feature in Phase 4 runs its own isolated LangGraph swarm built on `langgraph-swarm.create_swarm`. Features within the same dependency layer run concurrently — each in its own swarm instance, on its own thread, with its own memory context. There is no shared mutable state between concurrent swarms. The agents within a single swarm rotate via named handoff tools — there is no central dispatcher. Each agent reads the current graph state, decides what needs to happen next, and either acts or transfers control to the agent best positioned to continue.

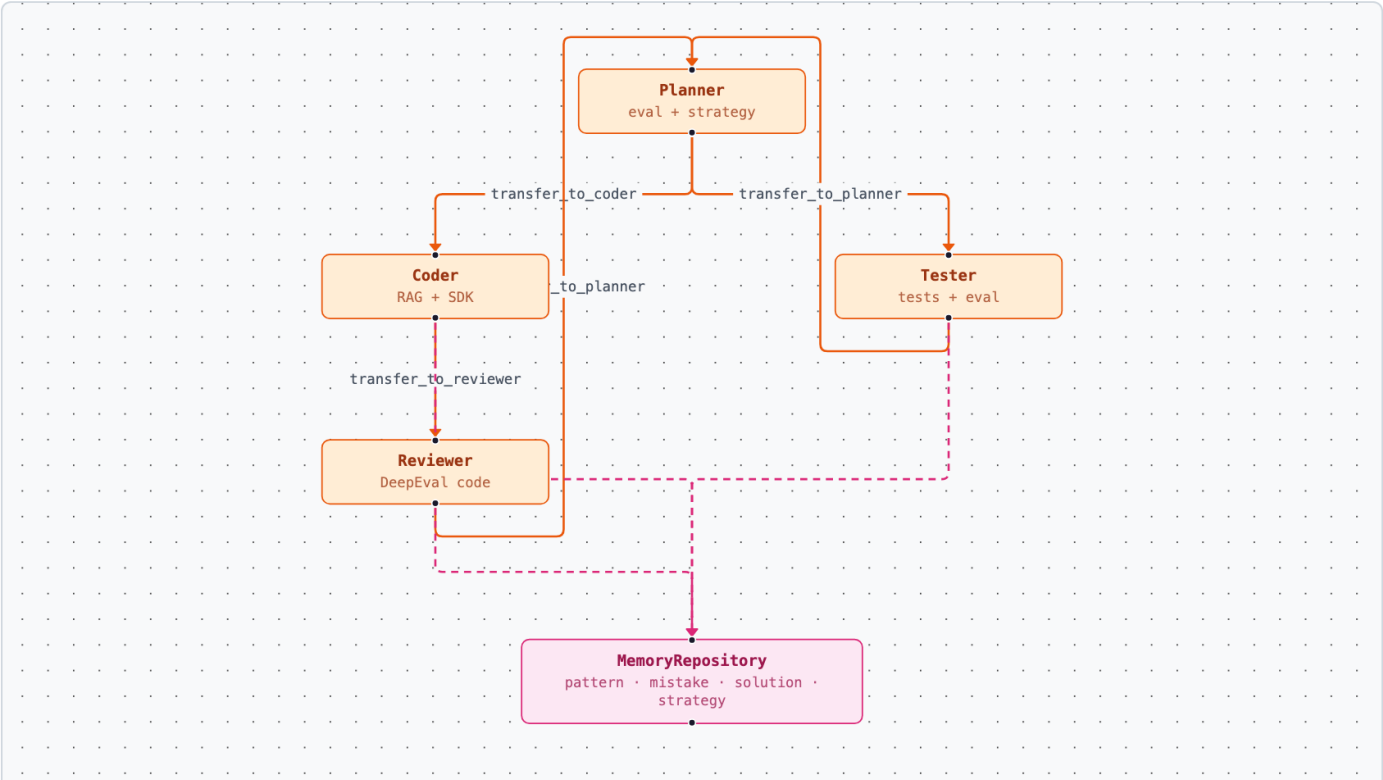


Figure 3 — Per-feature swarm handoff topology. Solid orange edges are LangGraph transfer tools; dashed pink edges are memory reads/writes.

THE FOUR AGENTS

Every swarm contains exactly the same four agents. Their roles are narrow by design — each agent has a focused system prompt, a restricted tool set, and a clear definition of done.

Planner	The entry point for every swarm invocation. Reads the spec in full, queries eval history for prior attempts on this feature, and recalls strategies from memory. Decides the opening move: delegate to the Coder if implementation is needed, transfer to the Tester if code exists but tests are missing, or mark the feature complete if both pass threshold. After each Reviewer or Tester round-trip, the Planner re-evaluates and either signs off or requests another pass. It is the only agent that can terminate the swarm successfully.
Coder	Responsible for producing the implementation. Before writing a single line, the Coder runs two Qdrant RAG queries — one for similar specs (to understand the expected interface) and one for similar code artifacts (to reuse proven patterns). It then recalls patterns and mistakes from memory. For straightforward tasks it writes code directly using its tool set; for complex implementations it delegates to a <code>claude_agent_codegen</code> deep agent that has full filesystem access and can iterate — run the linter, fix import errors, restructure files — before returning a result. The Coder then transfers to the Reviewer.

Reviewer	Evaluates the generated code against a DeepEval rubric covering correctness, coherence, security, and style. The Reviewer does not generate code — its role is adversarial evaluation. When it finds problems, it records them as Mistake nodes in memory (with root cause and file context) and records the fix as a paired Solution node. It then transfers back to the Planner with a structured verdict. Future Coder agents on similar features will recall these mistakes before writing their first line.
Tester	Writes a test suite against the spec's acceptance criteria — unit tests, integration tests, and edge cases. Evaluates its own output on correctness, coherence, and completeness using DeepEval. Records test failures and coverage gaps as Mistake nodes so future Testers on similar features start with that knowledge. Transfers back to the Planner with pass/fail status and a structured test report.

HANDOFF TOPOLOGY AND FLOW CONTROL

The handoff graph is intentionally not a strict pipeline. The Planner can send control to the Coder or the Tester independently — if tests already exist from a prior attempt, the Planner skips straight to the Tester rather than regenerating code. The Reviewer always returns to the Planner (never directly to the Coder) so that the Planner can weigh the review verdict against the remaining handoff budget before deciding whether another coding pass is warranted. This hub-and-spoke pattern around the Planner prevents runaway Coder-Reviewer cycles from consuming the entire budget on a single feature.

HANDOFF BUDGET

Every swarm is bounded by `max_codegen_handoffs` (default 50). Each agent-to-agent transfer consumes one unit. If the budget is exhausted before the Planner signs off, the orchestrator records the feature as failed, surfaces an incident, and moves to the next layer. A single failing feature never stalls the pipeline.

ADAPTIVE STRATEGY OVERRIDES

After each dependency layer completes, the orchestrator reviews the layer's aggregate pass rate. If it drops below the configured threshold, two overrides activate for all remaining features: the Coder is forced onto the deep-agent SDK path (disabling the direct-write shortcut), and the handoff budget is tightened. The premise is that when a layer underperforms, lightweight codegen is not keeping up with the complexity of the remaining specs — the SDK path's ability to iterate on disk and run the linter between attempts produces higher-quality output at the cost of more wall-clock time. When a subsequent layer's pass rate recovers, the overrides relax automatically.

MEMORY INTEGRATION WITHIN THE SWARM

Memory is not a post-processing step — it is woven into the hot path of every agent. Before the Coder writes its first line, it has already queried the shared memory graph for patterns relevant to this feature's capability tag and for mistakes recorded by prior Reviewers on similar code. Before the Planner picks a strategy, it has already recalled what worked on the last three runs of this feature. This means the swarm improves within a single run — a mistake recorded by the Reviewer on feature A is immediately available to the Coder on feature B when B's swarm starts, even before the run finishes.

CROSS-FEATURE LEARNING WITHIN A RUN

Because memory is written to Neo4j and Qdrant after each agent action — not batched at run end — features that start later in the dependency order benefit from lessons learned by features that ran earlier in the same run. The system gets smarter as a run progresses, not just across runs.

5. Procedural Memory

How the system learns across runs

Memory is the mechanism by which the system improves over time. It is split into two tiers — **semantic** memory (generalised lessons — what you should do) and **episodic** memory (specific past trajectories — what actually happened last time). Both tiers are stored in a dedicated Neo4j database with embeddings mirrored into Qdrant for hybrid semantic + keyword recall. Every agent reads memory before acting and writes memory after.

HOW PROCEDURAL MEMORY IS CREATED

Memory nodes are not manually authored — they are a byproduct of the swarm doing its job. Every time the Coder produces an implementation it considers reusable, it calls `record_pattern` to write a Pattern node describing the structure, the language idiom, and the capability context. Every time the Reviewer finds a problem — a broken import, an unsafe query, an inconsistent return type — it calls `record_mistake` with a root cause description and a reference to the offending file. If the fix is known, it calls `record_solution` to pair a Solution node with the Mistake. Every time the Planner makes a strategy decision — whether to use the SDK path, how to structure the output directory, which dependency to implement first — it calls `record_strategy`. These writes happen during the live swarm, not in a post-processing step, which means memory accumulates continuously across the run.

Episodic memories are created differently. After a feature swarm terminates — whether by success or budget exhaustion — the orchestrator synthesises the full event trajectory into a narrative summary and stores it as an Episode node. This synthesis uses the swarm's event log, the eval scores, and the IDs of every semantic memory recalled during the run. The result is a structured record of what happened, what worked, and which prior memories influenced the outcome.

MEMORY IS EARNED, NOT CONFIGURED

There are no manually defined rules, templates, or knowledge bases to maintain. The memory graph is entirely self-generated from agent observations during real runs. The first run on a new codebase starts with no memory; subsequent runs start progressively more informed. The system improves by doing, not by being told.

HOW PROCEDURAL MEMORY IS LEVERAGED

Before any agent takes its first action in a swarm, it issues a `recall_memories` call scoped to the current feature's capability tag. The call runs a hybrid Reciprocal Rank Fusion query — merging Neo4j keyword matches with Qdrant vector matches — and returns the top-ranked memories weighted by relevance score. The Coder receives Pattern nodes (reusable structures) and Mistake + Solution pairs (known failure modes with their fixes) before writing a single line of code. The Planner receives Strategy nodes (proven approach decisions) and Episode summaries (what happened on the last few runs of this exact feature) before choosing its opening move. The Reviewer and Tester receive Mistake nodes (patterns to watch for) before evaluating.

This recall step is not advisory — it directly shapes agent behaviour. A Coder that recalls "parameterised queries prevent injection in this codebase" will use parameterised queries without the Reviewer having to catch the violation. A Planner that recalls "the last two runs of the auth feature failed when using the direct-write path; the third succeeded with the SDK path" will pick the SDK path immediately. A Tester that recalls "edge-case tests for this API must include null payload variants" will include them without being told. Memory turns prior failure into current constraint, and prior success into current prior.

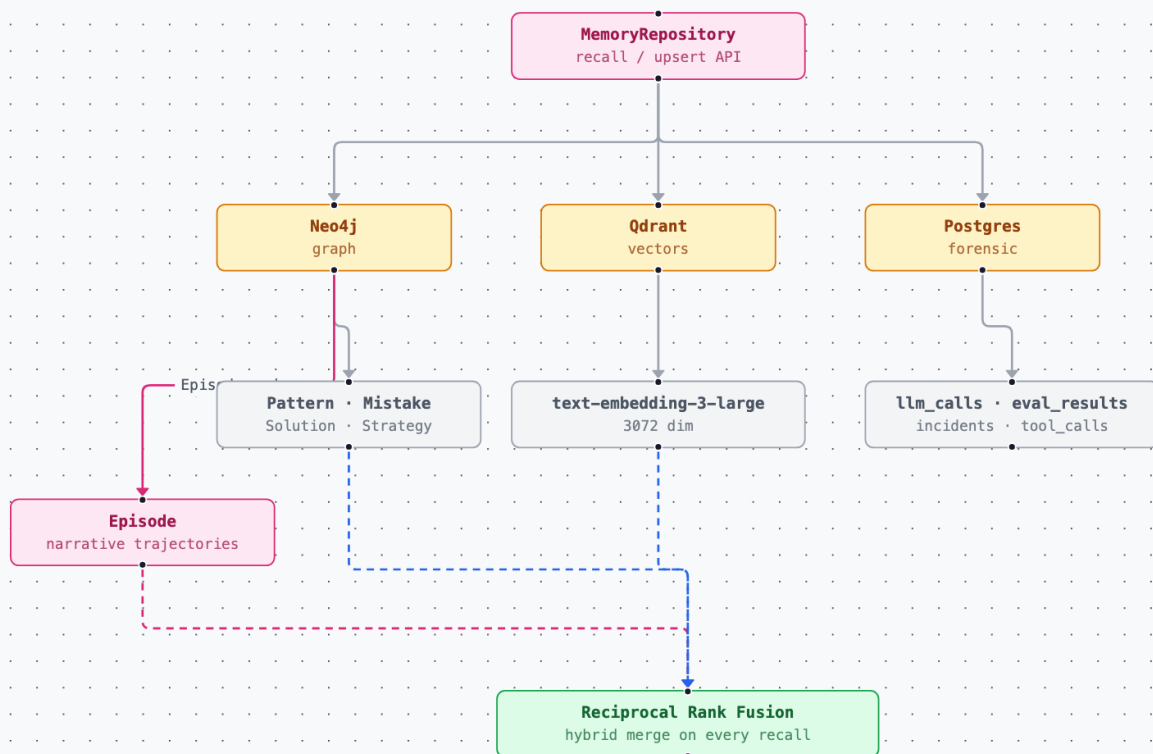


Figure 4 — Memory topology. Solid edges persist to a store; dashed edges feed the hybrid Reciprocal Rank Fusion merge used by every `recall_memories()` and `recall_episodes()` call. Pink nodes are the semantic tier + new episodic tier; blue is the hybrid merge.

SEMANTIC TIER

Four node types that encode generalised lessons — strip the temporal context and keep the teachable nugget.

Pattern	Reusable code structures the Coder recognises as good. Written by Coder, read by Coder.
Mistake	A concrete failure mode with a root cause. Written by Reviewer + Tester, read by all agents.
Solution	A fix that resolved a mistake. Paired with the Mistake node it fixes. Written by Reviewer + Tester, read by all agents.
Strategy	High-level approach decisions. Written by Planner, read by Planner in future runs.

EPISODIC TIER

After every feature swarm completes, the orchestrator synthesises a **narrative summary** (up to 300 words) of the trajectory plus **key turning-point events** (up to 15) (strategy picks, rejections, pivots, test passes). The result is stored as an `Episode` node in Neo4j linked to its `Run` via `PRODUCED_IN`, and embedded into the `dark_factory_episodes` Qdrant collection with enriched payloads (spec IDs, eval scores, recalled memory IDs).

Episodes now capture **which semantic memories influenced the outcome** — the synthesis prompt includes the IDs of every pattern, strategy, and prior episode recalled during the feature's lifecycle. This lets future Planners see not just “the last run succeeded” but “the last run succeeded *because it applied pattern-abc.*” `(Episode)-[:APPLIED]->(Pattern/Strategy/Episode)` graph edges enable traversals like “which episodes used this pattern?” and “which patterns come from successful runs?”

At the start of every feature, the Planner calls `recall_episodes(feature_name=<current>)` which runs the same hybrid Reciprocal Rank Fusion merge used by `recall_memories` — Neo4j keyword match on the summary text + Qdrant vector match on the embedding — returning the top-ranked past trajectories for the same feature. This gives the Dark Factory a **temporal reasoning layer** the semantic tier structurally cannot provide.

WHY EPISODIC WHEN YOU HAVE SEMANTIC?

Semantic memory answers “*what should I do?*” with lessons stripped of context; episodic memory answers “*what actually happened last time I was in this exact situation?*”. For a Planner picking between five possible strategies, the episodic record of which one worked on the last three runs of the same feature is often more actionable than a generalised Pattern hit. The two tiers complement each other — semantic for transfer learning across features, episodic for continuity within a feature.

FEEDBACK LOOP

When an agent recalls a memory and the resulting evaluation passes, the memory's relevance score is **boosted** (and `times_applied` incremented). When the evaluation fails, the memory is **demoted** (relevance decreases but `times_applied` is NOT incremented — a demote means the memory was recalled but didn't help). Every run decays all memory relevance by 5%. Relevance scores are synced to Qdrant after every boost, demote, and decay so vector search ranking stays consistent with Neo4j. This produces a natural forgetting curve — stale lessons fade, proven lessons strengthen, and contradictory lessons compete.

Recalled memory IDs are tracked in two sets: **per-eval** (cleared after each evaluation, used for boost/demote feedback) and **per-feature** (accumulated across the entire swarm lifecycle, used for episode synthesis). This ensures strategic memories recalled by the Planner get credit even when the evaluation happens several handoffs later in the Reviewer.

HYBRID RETRIEVAL + CROSS-FEATURE RECALL

The `recall_memories` tool uses Reciprocal Rank Fusion to merge Neo4j keyword matches with Qdrant vector matches — exact-string matches (like a specific error code) and semantic matches (like “similar to this situation”) in one call, weighted together.

Recall runs **two Qdrant passes**: one feature-scoped (high precision) and one unscoped (cross-feature). A pattern about parameterised SQL recorded by feature “auth” now surfaces when working on feature “user-profile.” When a Mistake is recalled, its associated Solution (via `RESOLVED_BY` edge) is returned inline so the agent sees the problem AND the fix in a single hit.

CROSS-FEATURE BRIEFING

Memory crosses feature boundaries *within the same run*— when feature B starts, the patterns and mistakes from feature A are already indexed and retrievable. The briefing excludes the current feature's own memories so retried features don't see redundant data. Agents don't have to wait for the next run to benefit from what the current run just learned.

MEMORY LIFECYCLE

The feedback loop drives memory evolution across runs: agents recall → use → eval → boost or demote → decay → prune → next run recalls with updated rankings.

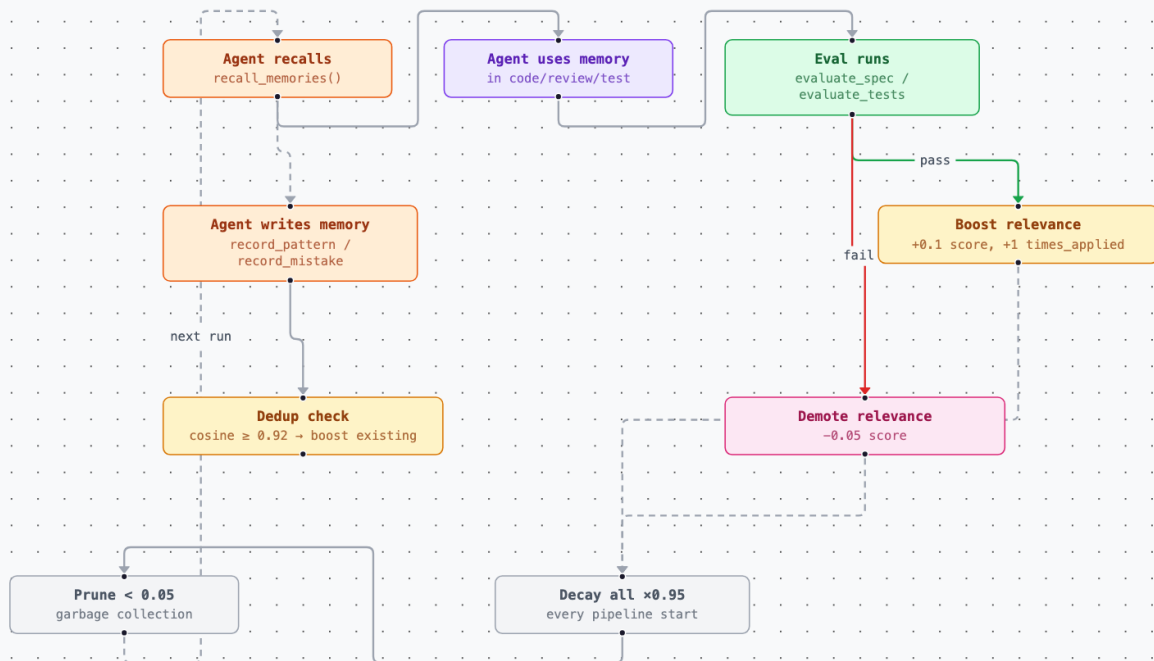


Figure — Memory lifecycle. Green edge = eval passed (boost). Red edge = eval failed (demote). Dashed = cross-run boundary.

MEMORY HYGIENE

Five mechanisms keep the graph clean. **Write-time dedup**: before creating a new memory, the repository embeds the candidate and cosine-matches against existing memories above `memory_dedup_threshold` (default 0.92). For Patterns and Strategies, dedup is **cross-feature** — the same pattern discovered by different features consolidates into one high-relevance node. Mistakes and Solutions stay feature-scoped to avoid conflating distinct failure modes. **Relevance-weighted Reciprocal Rank Fusion**: the hybrid recall merge multiplies each rank contribution by the memory's relevance score, so the feedback loop influences retrieval ordering. **Garbage collection**: `prune_low_relevance(threshold=0.05)` runs at every pipeline start, deleting memories that have decayed below usefulness from both Neo4j and Qdrant. **Qdrant payload sync**: relevance scores, run IDs, timestamps, and recall counts are kept in sync between Neo4j and Qdrant payloads. **Memory graph dashboard**: the Metrics tab has a Memory section with per-type node counts, relevance histograms, top-10 recalled memories, and a 7-day boost/demote effectiveness KPI row.

WHY CROSS-FEATURE DEDUP MATTERS

Without cross-feature dedup, a Coder recording “use parameterised queries” across five features creates five near-identical Pattern nodes that all rank high for SQL-related queries. The agent's recall list fills with paraphrases of the same idea and burns context. With cross-feature dedup, that single idea accumulates boost signal every time it's rediscovered, reaching high relevance faster and getting recalled first in every future run that needs it.

6. Observability

Three independent telemetry pipelines

Every meaningful event in the system is fanned out to three independent sinks. No single telemetry pipeline's failure blocks any other — the Prometheus counters fire even if Postgres is down; the ProgressBroker emits events to the UI even if Prometheus is disabled.

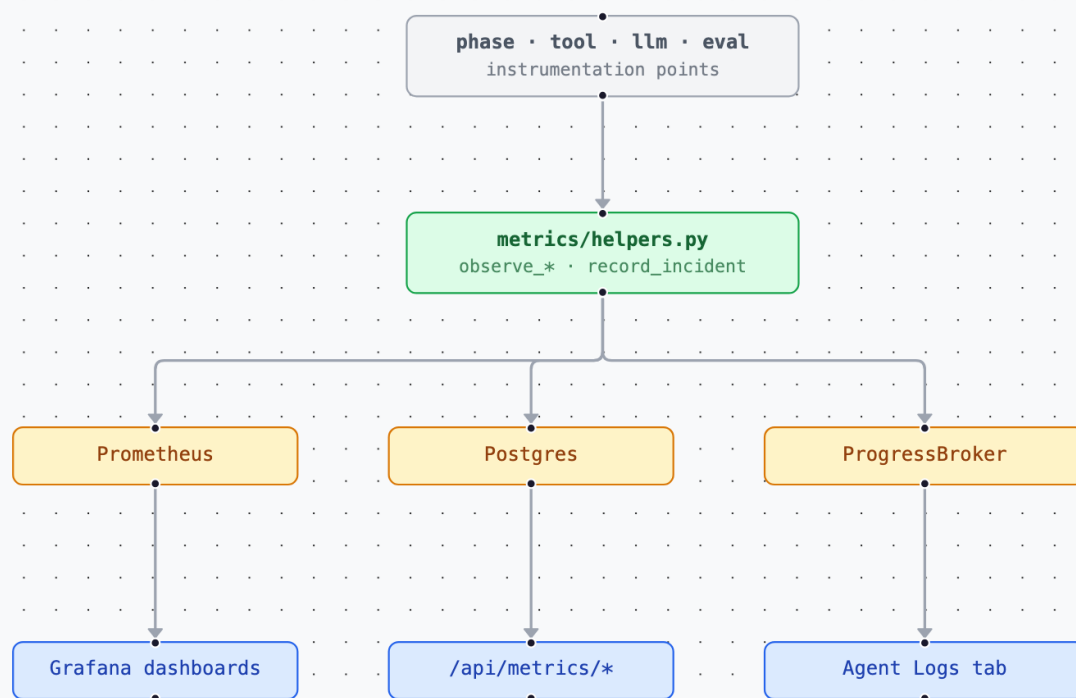


Figure 5 — Observability fan-out from instrumentation points through three independent sinks

Prometheus	Always-on in-process counters + histograms for runs, phases, tool calls, LLM invocations, reconciliation outcomes, deep-agent timeouts, and BackgroundLoop sampler ticks.
Postgres	Optional forensic store. Writes high-cardinality rows: every LLM call with token counts and cost, every eval result with score breakdowns, every tool call, every incident with a stack trace.
ProgressBroker	In-process pub/sub. Subscribers receive AG-UI events in real time — used by both the <code>/api/agent/events</code> Server-Sent Events stream and the server-side metric recorders.
Grafana	Pre-provisioned dashboards over Prometheus for pipeline throughput, cost rollups, quality trends, and incident budgets.
The Metrics tab in the UI surfaces multiple dashboards backed by <code>/api/metrics/*</code> endpoints. Those endpoints read from Postgres when available and fall back to Prometheus scrapes otherwise. RUN DETAIL POPUP Clicking any run ID in the Manufacture tab's history opens a dedicated popup window with these tabs:	
Agent Log	Historical progress events for the run — same color-coded badge layout as the main Agent Logs tab, with text filter and expandable JSON payload detail per event. Default tab on open.
Metrics	Status, pass rate, duration, spec/feature counts, LLM cost (broken down by pipeline phase), incidents, eval metrics, tool calls, artifacts, decomposition stats.
Evaluations	Per-spec evaluation tree with requirements, metric scores, attempt history.
Episodes	Episodic memory timeline — feature narratives, outcomes, key turning-point events, eval scores, recalled memory IDs.

Output	File explorer for generated code/artifacts with syntax highlighting. Download ZIP button streams the full output as a zip archive.
Compare	Side-by-side comparison against another run selected from a dropdown. Shows pass rate, duration, and feature-level status deltas with green/red coloring. 'View File Diffs' opens a unified diff viewer with line-level syntax coloring, powered by pre-computed MD5 manifests for efficient comparison.
Traceability	Requirements → specs → files → tests → eval scores matrix. Table/Graph toggle: table view shows expandable rows with bulleted evaluations, files (linked to S3 presigned URLs), and tests; graph view renders an interactive React Flow dependency graph with status-colored nodes (green=pass, red=fail, yellow=no evals). Info tooltip on non-pass statuses explains which specs/metrics failed.

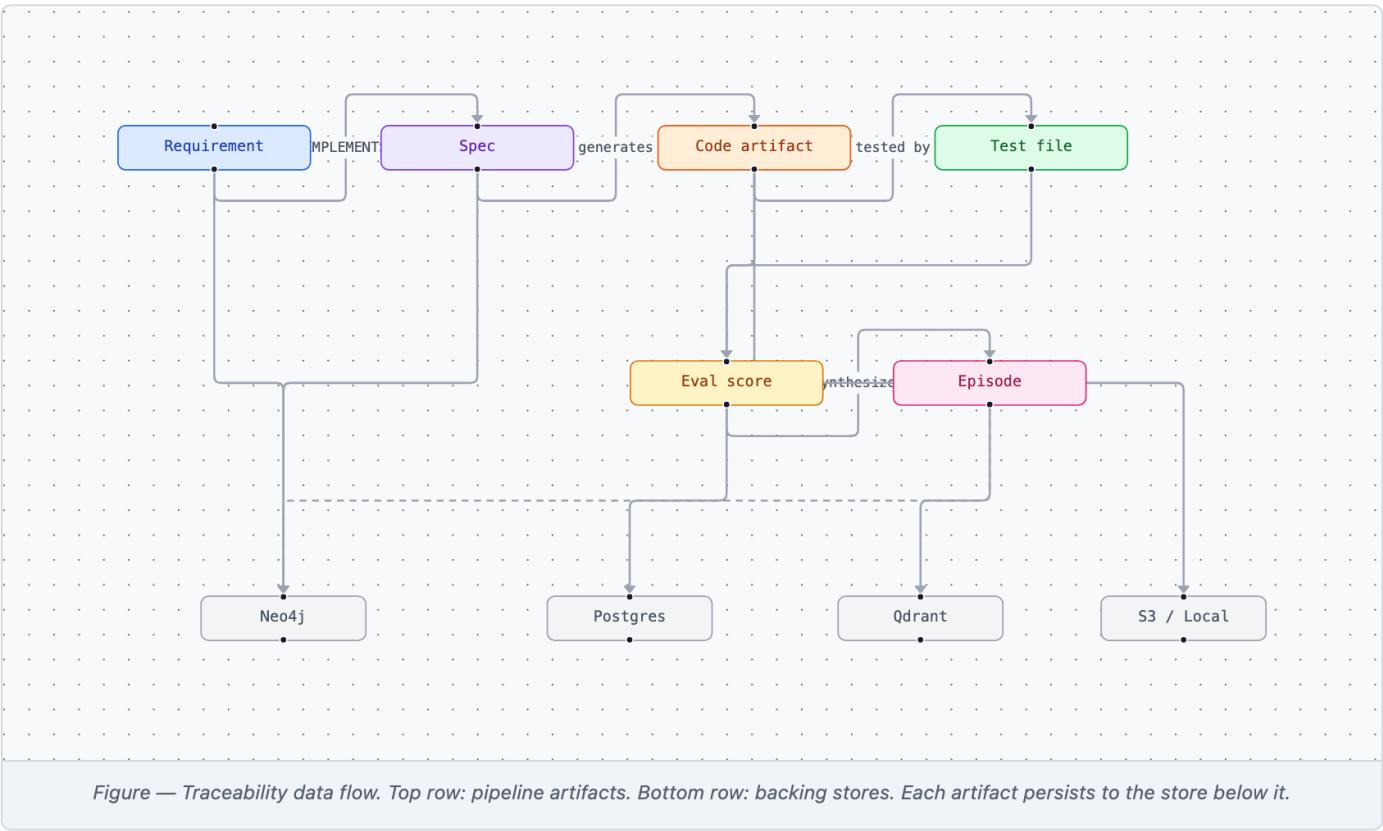
The Agent Log tab in Run Detail shares the same formatting code (`lib/agentLogFormat.ts`) as the main Agent Logs tab — event badges, color coding, and human-readable event descriptions are identical. The only difference is data source: the main tab reads from a live Server-Sent Events stream; the Run Detail tab loads historical records from the `progress_events` Postgres table via `GET /api/metrics/runs/{run_id}` .

6.5 Traceability & Storage

How data flows from requirements to S3

END-TO-END DATA FLOW

A single requirement traces through the entire system: Requirement → Spec (via IMPLEMENTS) → Code artifact (via swarm coder) → Test file (via swarm tester) → Eval score (via DeepEval judge) → Episode (via post-mortem synthesis). Each step persists to a different store.



STORAGE ARCHITECTURE

All pipeline output flows through `RunStorage` which delegates to the configured backend. When `STORAGE_BACKEND=local` and `S3_BUCKET` is set, a `ReplicatedStorage` writes to both local disk (fast) and S3 (durable). Reads fall back to S3 when local is empty (handles container restarts). MD5 hashes are computed at write time and stored in `.md5-manifest.json` for efficient cross-run diffing.

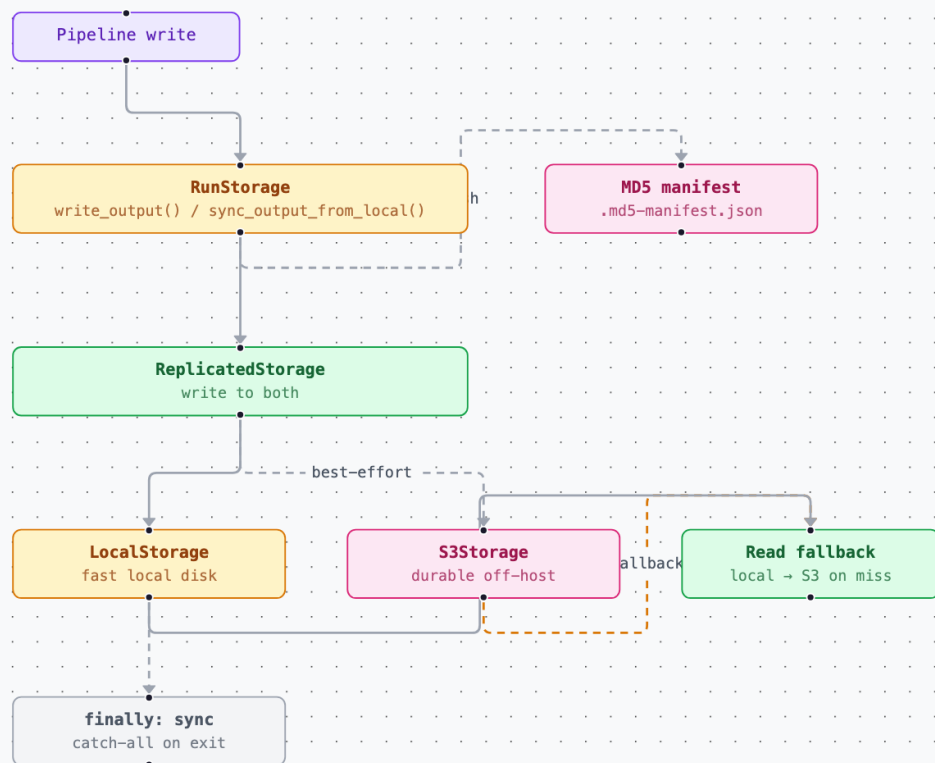


Figure — Storage architecture. Writes go to both backends; reads fall back to S3 on local miss. MD5 manifest enables $O(1)$ diff comparisons.

7. Cooperative Cancellation

Why the Cancel button actually works

Stopping a distributed multi-agent pipeline cleanly is deceptively hard. The naive approach (`thread.kill`, subprocess termination) leaves half-written Neo4j nodes, orphaned Qdrant embeddings, and never-flushed metric rows. AI Dark Factory uses **cooperative cancellation**: a single module-level `threading.Event` is polled at every hot-path checkpoint across all six phases.

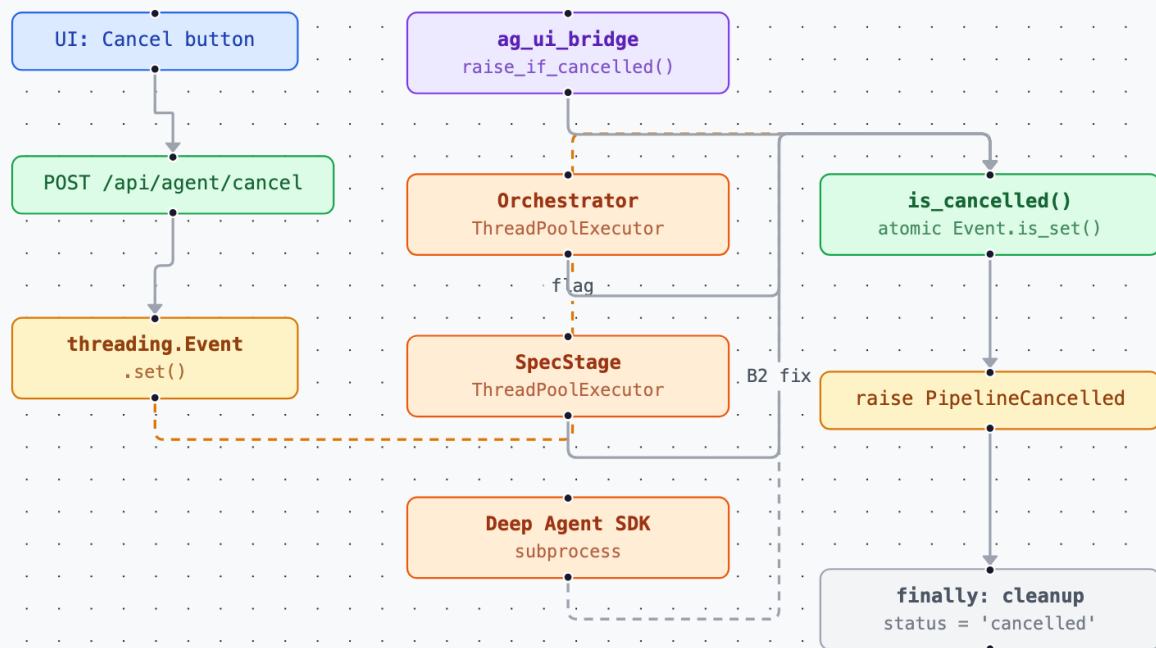


Figure 6 — Cancel signal propagation. The UI path (left) sets a flag; the worker path (right) polls it at every checkpoint.

Every pipeline phase, every tool call, every loop body calls `raise_if_cancelled()` at its start. When the flag is set, the call raises `PipelineCancelled`, which propagates up through each phase's finally blocks, gets caught by the AG-UI bridge's top-level handler, and is translated into a clean `status="cancelled"` run record — not a generic error. The flag is auto-reset at the start of every run so a cancel signal cannot bleed into the next invocation.

8. L4 Agentic Classification

Where this sits on the Vellum scale

This system operates at **L4 — Fully Autonomous / Explorer** on the [Vellum agentic behavior scale](#). The four L4 traits are all implemented:

Persistent state across sessions	Neo4j procedural memory + Qdrant embeddings + eval history + run history all survive restarts.
Refines execution from feedback	DeepEval scores drive memory boosts/demotes, adaptive thresholds, and mid-run strategy overrides.
Parallel execution	Concurrent feature swarms within each dependency layer, bounded by <code>max_parallel_features</code> .
Real-time adaptation	Layer-level strategy switches + cross-feature briefing within the same run.
Cross-feature reconciliation	Phase 5 extended Claude Agent SDK pass that sees the full output and polishes the integration.

9. Data Model Reference

Key Neo4j node labels and relationships

The Neo4j graph is the single source of truth for pipeline state. The primary labels and their roles:

Requirement	Parsed from input files. One per meaningful requirement. Carries id, description, source path.
Spec	Generated in Phase 2. Carries capability, scenarios, acceptance criteria, eval scores. Connected to its Requirement via IMPLEMENTS.
EvalResult	One per spec evaluation or artifact evaluation. Stores per-metric scores and the judge's reasoning.
Run	One per pipeline invocation. Stores spec_count, feature_count, pass_rate, mean_eval_scores, worst_features, duration, status.
Episode	Autobiographical record of a feature swarm's execution. Linked to Run via PRODUCED_IN, and to recalled memories via APPLIED edges.
Memory	Pattern / Mistake / Solution / Strategy nodes. Mistake-[:RESOLVED_BY]->Solution pairs the problem with its fix.
Key relationships: <code>Spec-[:IMPLEMENTS]->Requirement</code> , <code>Spec-[:DEPENDS_ON]->Spec</code> , <code>EvalResult-[:EVALUATED_IN]->Run</code> , <code>Episode-[:PRODUCED_IN]->Run</code> , <code>Mistake-[:RESOLVED_BY]->Solution</code> .	

10. Extensibility

Where to cut in

ADDING A NEW PIPELINE PHASE

Create a new module in `src/dark_factory/stages/` with a class exposing a `run()` method. Wire it into `src/dark_factory/api/ag_ui_bridge.py` between the existing phases, surrounding it with `StepStartedEvent` / `StepFinishedEvent` emissions and a `raise_if_cancelled()` checkpoint. Add a test file in `tests/`.

ADDING A NEW AGENT TO THE SWARM

Define a new agent in `src/dark_factory/agents/swarm.py` using `create_agent`, add handoff tools that name it, wire the new agent into the `create_swarm` list, and extend every other agent's system prompt to know when to transfer to it.

ADDING A NEW TOOL

Tools live in `src/dark_factory/agents/tools.py` and use the `@tool` decorator from LangChain. Add a new decorated function, then add its name to the `allowed_tools` list of the agent(s) that should be able to call it.

ADDING A NEW METRIC

Declare the Counter / Histogram in `src/dark_factory/metrics/prometheus.py` and add an `observe_*` helper. Call the helper from the relevant instrumentation point. If you also want a Postgres row, add a recorder method to `metrics/recorder.py`.